Demo app: pharmacy-api (Spring Boot)

Module 4: Automated Testing & Quality Gates

Day 3 · Azure DevOps, CI/CD and AI-Assisted Release Readiness

What We'll Cover

1. The automated test pyramid — unit, integration, end-to-end
2. Publishing test results & code coverage
3. Static analysis and quality gates — SonarQube / SonarCloud
4. Dependency & license scanning
5. Flaky-test handling
6. Test traceability back to work items
7. Defining pass/fail thresholds that block low-quality changes
8. Case study: wire a real quality gate into the pipeline

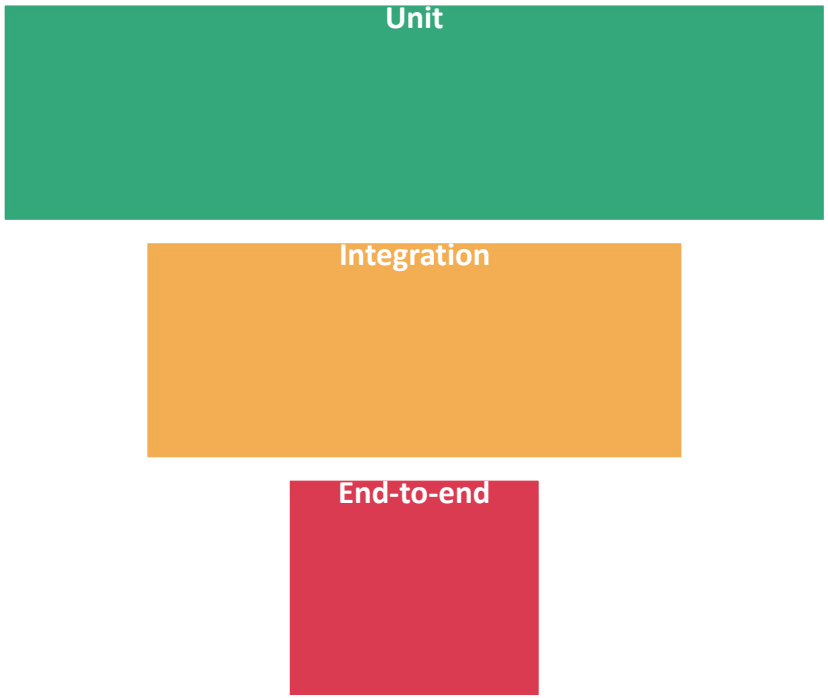
Where Module 3 Left Off

- candidate → Test gate → drop proved the pipeline, but “Test” was one pytest step
- That answers whether the code runs — not whether it's good enough to ship
- Module 4 opens up what belongs inside that gate before anything is trusted enough to promote
- Same pattern, deeper gate: more checks have to pass before drop exists

Today's question

If Test passes, is the build actually good enough to ship? What else should block it — coverage, code smells, vulnerable dependencies, an untracked change?

The Automated Test Pyramid



- One class/method, no I/O — many, fast, cheap to run on every push
- Component + its collaborators (DB, HTTP) — more, moderate speed
- Full system, real-ish environment — few, slow, expensive

More tests, lower in the pyramid — that's what makes the pipeline fast enough to run on every push.

Mapped Onto pharmacy-api

Unit — PrescriptionServiceTest

- Dispenses against stock, refill count, controlled-substance cap
- No Spring context, no network — runs in milliseconds
- Where the Lab 5-style boundary bug gets caught

Integration — PrescriptionControllerIT

- @SpringBootTest, real HTTP call into the controller
- Checks the whole request → service → repository path
- Slower — fewer of these, on purpose

End-to-end (not yet built)

- Would call a deployed instance over real HTTP
- Belongs after Module 5's Dev deploy exists
- Smoke test in Lab 5/Module 5 is this layer's first example

Publishing Test Results & Coverage

- pytest/JUnit write results as XML — the pipeline needs that file, not console output
- PublishTestResults@2 reads it and renders pass/fail per test on the run summary
- condition: succeededOrFailed() — publish results even when tests fail, so the failure is visible, not hidden
- JaCoCo (Java) / coverage.xml produces a coverage report the same way — published separately with PublishCodeCoverageResults@2

In pharmacy-api

jacoco-maven-plugin runs on `mvn test`, writing target/site/jacoco/jacoco.xml — the exact file both PublishCodeCoverageResults@2 and SonarCloud read.

Common mistake

Forgetting succeededOrFailed() on the publish step — a failing run then shows no test detail at all, just a red X.

Code Coverage: What It Actually Tells You

What a coverage number DOES tell you

- Which lines ran during the test suite
- Where there's zero test exposure at all
- A trend — is coverage slipping as the codebase grows

What it does NOT tell you

- That the test actually asserted anything meaningful
- That the right boundary/edge case was tested
- That the code is correct — a line can run and still be wrong

This is why Module 4's gate pairs coverage with SonarCloud's code-quality checks — a number alone isn't the proof.

Static Analysis & Quality Gates

- SonarQube = self-hosted; SonarCloud = the same analysis, hosted by Sonar — both plug into Azure Pipelines via official tasks
- SonarCloudPrepare → your build/test step → SonarCloudAnalyze → SonarCloudPublish
- Checks: bugs, vulnerabilities, code smells, duplication, and coverage — specifically on new/changed code, not just the whole codebase
- A Quality Gate is a named set of pass/fail conditions on those metrics — SonarCloudPublish can fail the pipeline if the gate fails

Default gate (Sonar way)

- 0 new bugs
- 0 new vulnerabilities
- Coverage on new code ≥ 80%
- Duplicated lines on new code < 3%

Focusing on NEW code (not the whole legacy base) is deliberate — it's enforceable without a giant one-time cleanup.

Dependency & License Scanning

- Your code can be clean and still ship a known-vulnerable library underneath it
- Dependency scanning checks every declared dependency against known-vulnerability databases (CVE/NVD)
- License scanning checks the licenses those dependencies carry against what your organisation is allowed to redistribute
- Both run as a pipeline step — OWASP Dependency-Check, Mend, GitHub Advisory / Dependabot, or an Azure Marketplace extension

Why it matters here

pharmacy-api handles controlled-substance data. A vulnerable dependency or an incompatible license isn't just a bug — it's a compliance finding. This is the healthcare-domain reason this check exists at all, not a generic best practice.

Flaky-Test Handling

- A flaky test passes and fails on the same code, no changes — usually timing, shared state, or an external dependency
- The real risk isn't the flake itself — it's learners (and teams) learning to ignore red builds
- Options, worst to best: re-run until green (hides it), retry-on-failure with a hard cap, quarantine with a tracked ticket and a deadline, or fix the root cause
- Azure Pipelines: rerunFailedTests on the test task, or a dedicated ‘quarantine’ test category excluded from the blocking run but still tracked

Teaching point

Ask learners: if a test is flaky, is skipping it safer or more dangerous than leaving it blocking? (It's more dangerous — silent gaps are worse than a visible red build.)

Test Traceability Back to Work Items

- A test exists because a requirement or bug fix demanded it — traceability is the link proving that
- Azure DevOps: link a commit/PR to a work item (#1234 in the commit message or PR description), and Test Plans can link a manual/automated test case to that same item
- On audit, traceability answers: which requirement does this test verify, and which test proves this bug was actually fixed?
- This matters more in regulated domains — a pharmacy/health system often needs exactly this chain for compliance

In the labs

The Lab 5-style fix (Module 3) linked its branch to a work item ID. Module 4 carries that forward: the fix for the refill boundary bug gets linked the same way.

Defining Pass/Fail Thresholds

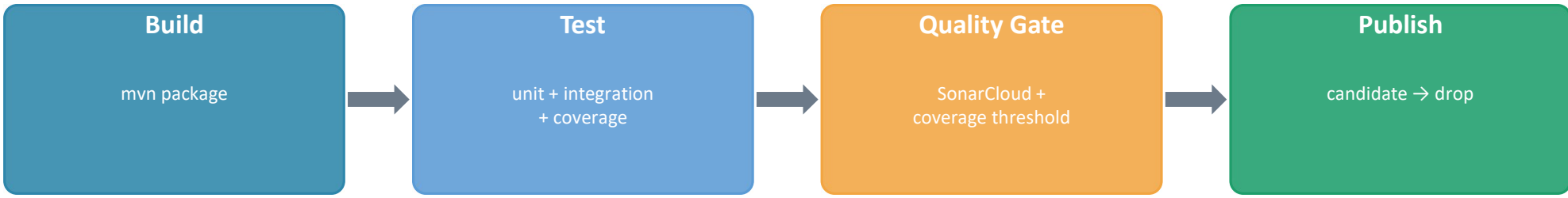
- A threshold only works if it's specific, measured automatically, and actually blocks — not just reported
- Coverage threshold: jacoco-maven-plugin's check goal can fail the Maven build itself below a set ratio
- Quality gate threshold: SonarCloudPublish can fail the pipeline task when the gate fails
- Both are enforced in code, in the repo, not as a rule someone has to remember to check manually

Where it sits in the pipeline

Build → Test (unit + integration, coverage collected) → Quality Gate (SonarCloud + coverage check) → only then Publish. Same shape as Module 3's candidate → gate → drop — just a deeper gate.

In pharmacy-api, the floor was set at 45% — not a round 80% — because that's what the project's real baseline coverage actually was. The threshold has to match reality, or it's noise, not a gate.

Putting It Together: The Module 4 Pipeline



A failing Quality Gate stops the run right there — Publish never executes, exactly like a failing Test stage in Module 3.

Today's Case Study

Wire quality gates into the pharmacy-api pipeline

- Publish JUnit test results and JaCoCo coverage on every run
- Add a SonarCloud scan and a quality gate to the pipeline
- Add a coverage threshold that can fail the build on its own
- Prove it: deliberately trip the gate, watch it block, then fix it and watch it pass

Full step-by-step is in the separate Module 4 Labs doc — this slide is the "why," not the "how."

Knowledge Check

A pipeline's coverage check uses jacoco-maven-plugin's `check` goal bound to the `verify` phase with a 70% LINE/COVEREDRATIO minimum. The build currently sits at 65% line coverage. What happens?

A. Nothing; coverage is informational only, it never fails a build

B. mvn verify fails, so any pipeline step running through verify fails with it

C. Only the SonarCloud quality gate would catch this, not Maven itself

D. It fails only if PublishCodeCoverageResults@2 is also missing from the pipeline

Answer

B — Correct

The coverage-threshold execution is bound to the verify phase, so mvn verify itself fails — it's a Maven build failure, not a separate Sonar-only check. Any pipeline step that runs mvn verify (or a later phase) fails with it, before SonarCloud even gets a say.

- A is wrong — that's true only if no threshold is configured at all
- C is wrong — Maven's own check goal already blocks it, independently of Sonar
- D is wrong — publishing results is a reporting step; it doesn't change whether the build itself passed

Key Takeaways

- The pyramid's shape (many fast unit tests, few slow end-to-end ones) is what keeps a quality gate affordable to run on every push
- Coverage is a floor, not a verdict — pair it with static analysis, not instead of it
- A threshold that doesn't actually fail the build is a dashboard, not a gate
- Dependency and license scanning exist because clean code can still ship a compromised or non-compliant dependency
- Traceability and flaky-test discipline are what make the gate trustworthy enough that people stop overriding it

Next: Module 5

Continuous Delivery, Environments & Deployment Strategies